

# Chapter 2: Built-in Data Structures Under the Hood

**Core Learning Objective:** Peel back the syntax of Python lists, dictionaries, tuples, sets, and deques to understand their underlying C structures, memory layout, dynamic resizing algorithms, and amortized Big-O complexities.

## 1. Python Lists: Contiguous Pointer Arrays

**Zero-Prerequisite Intuition: The "Coat-Check Ticket Rack" Metaphor** What is a Python `list` under the hood?

Imagine you run the coat-check room at a luxury theatre:

- Guest 1 checks in a giant heavy winter fur coat.
- Guest 2 checks in a tiny pair of sunglasses.
- Guest 3 checks in an umbrella.

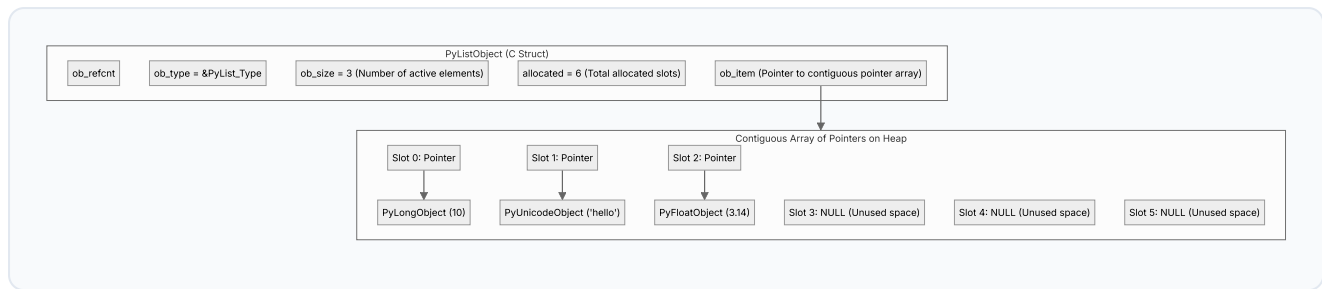
You cannot glue the fur coat, the sunglasses, and the umbrella together in a straight line—they are completely different sizes and shapes!

What do you do instead? You have a neat, orderly rack of **identical wooden hangers numbered 0, 1, 2, 3...**

- On Hanger #0, you hang a claim ticket pointing to the winter fur coat in the back room.
- On Hanger #1, you hang a claim ticket pointing to the sunglasses.
- On Hanger #2, you hang a claim ticket pointing to the umbrella.

In Python, a `list` does **NOT** hold the actual data items! A Python `list` is simply an **orderly rack of identical claim tickets (memory pointers)**. Each hanger is exactly 8 bytes wide, and each hanger points to wherever that object is floating on the warehouse floor (the Heap)!

A Python `list` is **not a linked list**. It is a **dynamically sized, contiguous array of pointers** (`PyObject**`).



## The CPython Definition ( `listobject.h` )

### Dynamic Resizing & Over-Allocation Formula

When you call `list.append()`, CPython checks if `ob_size == allocated`. If the buffer is full, it resizes the array using this exact formula:

$$\text{new\_allocated} = \text{new\_size} + (\text{new\_size} \gg 3) + (\text{new\_size} < 9 ? 3 : 6)$$

#### Key Takeaways:

- As the list grows, capacity increases in bursts:  $0 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 24 \rightarrow 32 \dots$
- Because resizing occurs geometrically, appending has an **amortized time complexity of  $O(1)$** , even though an individual resize operation costs  $O(N)$  memory copies.

### Time Complexity Matrix: List Operations

| Operation                       | Average Case | Worst Case           | Memory / Mechanism                                                  |
|---------------------------------|--------------|----------------------|---------------------------------------------------------------------|
| <code>list.append(x)</code>     | $O(1)$       | $O(N)$ (rare resize) | Appends to end of array                                             |
| <code>list.pop()</code>         | $O(1)$       | $O(1)$               | Decrements <code>ob_size</code>                                     |
| <code>list.insert(0, x)</code>  | $O(N)$       | $O(N)$               | Shifts all $N$ pointers right by 1                                  |
| <code>list.pop(0)</code>        | $O(N)$       | $O(N)$               | Shifts all $N - 1$ pointers left by 1                               |
| <code>list[i]</code> (Lookup)   | $O(1)$       | $O(1)$               | Direct pointer arithmetic: <code>ob_item + i * sizeof(void*)</code> |
| <code>x in list</code> (Search) | $O(N)$       | $O(N)$               | Linear iteration over pointers                                      |

## 2. Python Tuples: Fixed Memory Footprint

A `tuple` is an **immutable contiguous pointer array**. Because its length is fixed at instantiation, CPython eliminates the `allocated` capacity field.

### The Tuple Free List Optimization

To accelerate code execution, CPython maintains an internal **free list** for small tuples (up to 20 elements). When a tuple is destroyed, its memory is not immediately freed back to the OS; instead, it is retained in a freelist to instantly satisfy future tuple allocations without calling `malloc()`.

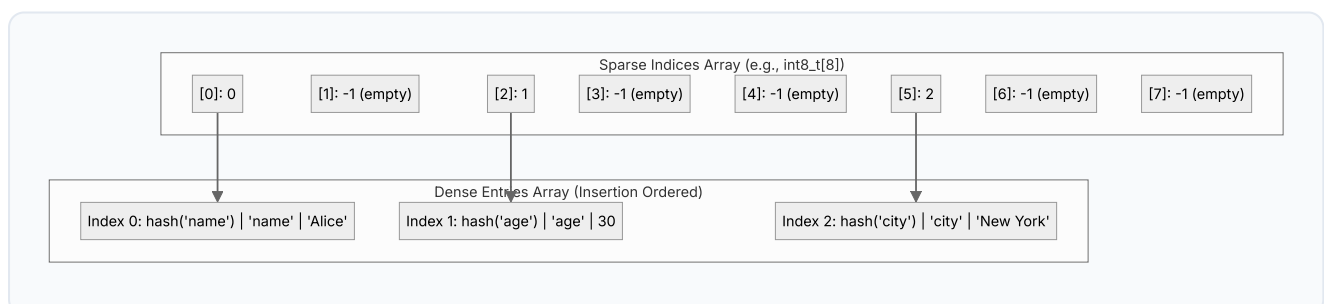
## 3. The Modern Dictionary: Compact Hash Tables

Prior to Python 3.6, dictionaries used a sparse hash table where each entry took 24 bytes (hash, key pointer, value pointer), resulting in vast amounts of wasted unused memory:

### The Raymond Hettinger Compact Dict (Python 3.6+)

Modern Python separates the sparse table into **two distinct arrays**:

1. **Indices Array (Sparse)**: An array of tiny integers (1 byte for small dicts: `int8_t`).
2. **Entries Array (Dense)**: A compact array storing only active entries in order of insertion.



### Why Dicts Preserve Insertion Order

Because new keys are simply appended to the end of the **Dense Entries Array**, iterating over a dictionary merely walks the entries array sequentially. This is why Python dictionaries are guaranteed to be insertion-ordered!

## Hash Collision Resolution: Perturbation Probing

When two keys produce identical hash table indices, CPython resolves collisions via **open addressing** with a pseudo-random perturbation sequence:

$$\text{perturb} \gg= 5$$

$$j = (5 \times j + 1 + \text{perturb}) \pmod{\text{mask}}$$

This probe sequence ensures that all slots in the table are systematically visited without clustering.

## Hashability Requirements

For an object to be used as a dictionary key or set element, it must satisfy the **Hash Contract**:

1. Implement `__hash__()` returning an integer that never changes during the object's lifetime.
2. Implement `__eq__()` such that if `a == b`, then `hash(a) == hash(b)`.
3. If an object is mutable, modifying it would alter its hash and break table lookups. Hence, mutable types (`list`, `dict`, `set`) unhashable: `TypeError: unhashable type: 'list'`.

## 4. Sets and Frozensets

A `set` in CPython is essentially a dictionary where the entries contain only keys and no values.

- **Lookup / Insert / Delete:**  $O(1)$  average.
- **Set Operations:**
  - Union (`A | B`):  $O(\text{len}(A) + \text{len}(B))$
  - Intersection (`A & B`):  $O(\min(\text{len}(A), \text{len}(B)))$
  - Difference (`A - B`):  $O(\text{len}(A))$
  - Symmetric Difference (`A ^ B`):  $O(\text{len}(A) + \text{len}(B))$

## 5. High-Performance Alternatives: `collections` & `heapq`

### `collections.deque` (Double-Ended Queue)

When you need to push and pop from both sides, never use `list.insert(0, x)` or `list.pop(0)`. Use `collections.deque`.

- In CPython, `deque` is implemented as a **doubly-linked list of fixed-size blocks** (each block holds 64 elements).
- Adding or removing from either head or tail is strictly  $O(1)$  **without resizing copies**.

## heapq : Priority Queues

heapq implements a **min-heap** directly on top of standard Python lists using 0-based array indexing:

- For node at index  $i$ :
  - Left child:  $2i + 1$
  - Right child:  $2i + 2$
  - Parent:  $\lfloor (i - 1) / 2 \rfloor$
- Operations:
  - `heapify(lst)` : Transforms list into heap in-place in  $O(N)$  time (sift-down algorithm).
  - `heappush(heap, item)` :  $O(\log N)$ .
  - `heappop(heap)` :  $O(\log N)$ .

## 6. Comprehensive Big-O Reference Matrix

| Data Structure     | Access by Index   | Search by Value   | Insert at Head | Insert at Tail   | Delete at Head  | Delete at Tail |
|--------------------|-------------------|-------------------|----------------|------------------|-----------------|----------------|
| <code>list</code>  | $O(1)$            | $O(N)$            | $O(N)$         | $O(1)$ amortized | $O(N)$          | $O(1)$         |
| <code>deque</code> | $O(N)$            | $O(N)$            | $O(1)$         | $O(1)$           | $O(1)$          | $O(1)$         |
| <code>dict</code>  | N/A               | $O(1)$ key lookup | N/A            | $O(1)$ amortized | N/A             | $O(1)$         |
| <code>set</code>   | N/A               | $O(1)$ membership | N/A            | $O(1)$ insert    | N/A             | $O(1)$ pop     |
| <code>heapq</code> | $O(1)$ (min item) | $O(N)$            | N/A            | $O(\log N)$ push | $O(\log N)$ pop | N/A            |

## Practice Problems & Interview Drills

---

### Drill 1: The `[[[]] * 5]` Multiplication Trap

What bug does this code introduce, and how does pointer layout explain it?

**Explanation & Solution:** The outer `* 3` does not clone the inner list; it copies the **pointer** to that same single list three times!

### Drill 2: Implement an LRU Cache from Scratch

*Requirement:* Implement a Least Recently Used (LRU) Cache with  $O(1)$  `get()` and  $O(1)$  `put()`. Do not use `collections.OrderedDict`.

**Architecture:** Combine a **hash map** (for  $O(1)$  key-to-node lookup) with a **doubly linked list** (for  $O(1)$  node removal and insertion to the front).

## Version Notes

---

- Dict insertion order is a language guarantee from 3.7; the `|` merge operator for dicts needs 3.9.
- `itertools.batched` (3.12) replaces hand-written chunking helpers like the one in the Chapter 01 exercise; `functools.cache` needs 3.9.

## Exercises

---

Three graded exercises for this chapter (two coding, one debugging) with hidden tests:

Attempt first; the reference solutions are in `exercises/_answers/ch02.py`.

## Further Reading

---

- [Data structures tutorial](#)
- [collections module](#)
- [TimeComplexity wiki](#)

---

## Check Yourself

---

Answer in your head or on paper first, then open each answer.

- ▶ 1. What is the average time complexity of `x` in `some_set` versus `x` in `some_list`?
- ▶ 2. Why must dict keys be hashable, and what rule links `__hash__` and `__eq__`?
- ▶ 3. Why is `list.insert(0, x)` slow but `collections.deque.appendleft(x)` fast?
- ▶ 4. Since which version do dicts preserve insertion order as a language guarantee?